

LOG-BASED BUG PATTERN MINING

IMPORTING THE LIBRARIES

```
import numpy as np
import pandas as pd
```

Loading the Dataset

```
df_log_based_bug_pattern_mining_dataset =
pd.read_csv('synthetic_log_dataset (1).xls')
```

Basic Dataset Overview

```
df_log_based_bug_pattern_mining_dataset.shape

(12500, 9)

df_log_based_bug_pattern_mining_dataset.head()

{"summary":{"\n  \"name\":
  \"df_log_based_bug_pattern_mining_dataset\", \n  \"rows\": 12500, \n
  \"fields\": [\n    {\n      \"column\": \"log_id\", \n
  \"properties\": {\n      \"dtype\": \"number\", \n      \"std\":
  3608, \n      \"min\": 1, \n      \"max\": 12500, \n
  \"num_unique_values\": 12500, \n      \"samples\": [\n
  1767, \n      11920, \n      8910 \n      ], \n
  \"semantic_type\": \"\", \n      \"description\": \"\" \n    } \n
  }, \n    {\n      \"column\": \"timestamp\", \n
  \"properties\": {\n      \"dtype\": \"object\", \n
  \"num_unique_values\": 12500, \n      \"samples\": [\n
  \"2025-09-01 00:30:47.474558\", \n      \"2025-09-01
  03:27:40.793743\", \n      \"2025-09-01 02:35:31.594535\" \n
  ], \n      \"semantic_type\": \"\", \n
  \"description\": \"\" \n    } \n    }, \n    {\n      \"column\":
  \"service_name\", \n      \"properties\": {\n      \"dtype\":
  \"category\", \n      \"num_unique_values\": 7, \n      \"samples\":
  [\n      \"data_processor\", \n      \"payment_api\", \n
  \"user_service\" \n      ], \n      \"semantic_type\": \"\", \n
  \"description\": \"\" \n    } \n    }, \n    {\n      \"column\":
  \"log_level\", \n      \"properties\": {\n      \"dtype\":
  \"category\", \n      \"num_unique_values\": 5, \n      \"samples\":
  [\n      \"ERROR\", \n      \"FATAL\", \n      \"INFO\" \n      ], \n
  \"semantic_type\": \"\", \n      \"description\": \"\" \n    } \n    },
  {\n      \"column\": \"log_message\", \n
  \"properties\": {\n      \"dtype\": \"category\", \n
  \"num_unique_values\": 515, \n      \"samples\": [\n      \"Query
```



```

---
0  log_id      12500 non-null int64
1  timestamp   12500 non-null object
2  service_name 12500 non-null object
3  log_level    12500 non-null object
4  log_message  12500 non-null object
5  error_type   12500 non-null object
6  stack_trace  2479 non-null object
7  build_id     12500 non-null object
8  status       12500 non-null object
dtypes: int64(1), object(8)
memory usage: 879.0+ KB

```

Interpretation

- We have 12500 observation and 9 attributes
- We have 1 numerical columns and 8 categorical columns
- Memory used is 879.0+ KB

Null value handling

```

df_log_based_bug_pattern_mining_dataset.isnull().sum()

log_id      0
timestamp   0
service_name 0
log_level    0
log_message  0
error_type   0
stack_trace 10021
build_id     0
status       0
dtype: int64

```

Interpretation

- We have 10021 null values in stack_trace column

```

# We'll replace the missing values in 'stack_trace' with an empty string.
# .fillna('') replaces all 'NaN' values with an empty text string,
# which makes it easier to combine the columns later.
df_log_based_bug_pattern_mining_dataset['stack_trace'] =
df_log_based_bug_pattern_mining_dataset['stack_trace'].fillna('')

df_log_based_bug_pattern_mining_dataset.isnull().sum()

log_id      0
timestamp   0
service_name 0

```

```
log_level      0
log_message    0
error_type     0
stack_trace    0
build_id       0
status         0
dtype: int64
```

Interpretation

- After replacing , we have 0 null values in the data set

```
print("\nDistribution of Error Types")
print(df_log_based_bug_pattern_mining_dataset['error_type'].value_counts())
```

Distribution of Error Types

```
error_type
NoError          10021
NullPointerException    990
IndexOutOfBoundsException  401
FileNotFoundException    377
IllegalArgumentException  256
TimeoutException      233
ConnectionResetException  222
Name: count, dtype: int64
```

```
# We'll create a new column called 'full_text' by combining
# 'log_message' and 'stack_trace'.
# This new column contains all the text information related to each
# log entry.
df_log_based_bug_pattern_mining_dataset['full_text'] =
df_log_based_bug_pattern_mining_dataset['log_message'] + ' ' +
df_log_based_bug_pattern_mining_dataset['stack_trace']

# Convert all text in the 'full_text' column to lowercase.
# This is important for consistency, so a model treats
# 'NullPointerException' and 'nullpointerexception' the same.
df_log_based_bug_pattern_mining_dataset['full_text'] =
df_log_based_bug_pattern_mining_dataset['full_text'].str.lower()

# Drop the original text columns from our DataFrame to keep it clean.
# 'axis=1' specifies that we are dropping a column, not a row.
df_log_based_bug_pattern_mining_dataset =
df_log_based_bug_pattern_mining_dataset.drop(columns=['log_message',
'stack_trace'])

# Display the prepared DataFrame to see the changes.
print("\n Prepared DataFrame (after cleaning) ")
df_log_based_bug_pattern_mining_dataset.head()
```

Prepared DataFrame (after cleaning)

```
{
  "summary": {
    "name": "df_log_based_bug_pattern_mining_dataset",
    "rows": 12500,
    "fields": [
      {
        "column": "log_id",
        "properties": {
          "dtype": "number",
          "std": 3608,
          "min": 1,
          "max": 12500,
          "num_unique_values": 12500,
          "samples": [
            1767, 11920, 8910
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "timestamp",
        "properties": {
          "dtype": "object",
          "num_unique_values": 12500,
          "samples": [
            "2025-09-01 00:30:47.474558", "2025-09-01 03:27:40.793743", "2025-09-01 02:35:31.594535"
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "service_name",
        "properties": {
          "dtype": "category",
          "num_unique_values": 7,
          "samples": [
            "data_processor", "payment_api", "user_service"
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "log_level",
        "properties": {
          "dtype": "category",
          "num_unique_values": 5,
          "samples": [
            "ERROR", "FATAL", "INFO"
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "error_type",
        "properties": {
          "dtype": "category",
          "num_unique_values": 7,
          "samples": [
            "NoError", "NullPointerException", "IndexOutOfBoundsException"
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "build_id",
        "properties": {
          "dtype": "string",
          "num_unique_values": 12400,
          "samples": [
            "build-392-658", "build-607-790", "build-876-807"
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "status",
        "properties": {
          "dtype": "category",
          "num_unique_values": 2,
          "samples": [
            "FAILURE", "SUCCESS"
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "full_text",
        "properties": {
          "dtype": "category",
          "num_unique_values": 2958,
          "samples": [
            "filenotfoundexception: illegal argument passed to function filenotfoundexception occurred at search_engine.service(service.java:280) at search_engine.util(util.java:195) at search_engine.util(util.java:188)"
          ]
        }
      ]
    }
  }
}
```

```

search_engine.main(main.java:342)\\n",\n
\"indexoutofboundsexception: connection reset by peer
indexoutofboundsexception occurred\\n    at
auth_service.util(util.java:120)\\n    at
auth_service.main(main.java:322)\\n    at
auth_service.service(service.java:80)\\n    at
auth_service.controller(controller.java:321)\\n\"\\n    ],\n
\"semantic_type\": \"\", \n    \"description\": \"\"\\n    }\\n
    }\\n ]\\n
n}","type":"dataframe","variable_name":"df_log_based_bug_pattern_minin
g_dataset"}

print("\\n Final DataFrame Info ")
df_log_based_bug_pattern_mining_dataset.info()

```

```

Final DataFrame Info
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12500 entries, 0 to 12499
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   log_id           12500 non-null  int64
1   timestamp        12500 non-null  object
2   service_name     12500 non-null  object
3   log_level        12500 non-null  object
4   error_type       12500 non-null  object
5   build_id         12500 non-null  object
6   status           12500 non-null  object
7   full_text        12500 non-null  object
dtypes: int64(1), object(7)
memory usage: 781.4+ KB

```

Feature Engineering (TF-IDF)

```

# Import the TfidfVectorizer from scikit-learn.
# 'scikit-learn' is the most popular library for machine learning in
Python.
from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize the TF-IDF Vectorizer.
# 'TfidfVectorizer' is a tool that does all the TF-IDF calculations
for you.
# We set a few parameters to improve performance:
# - max_features=5000: This tells the vectorizer to only consider the
5000 most frequent words.
# This helps reduce the size of our dataset and focuses on the most
important terms.
# - stop_words='english': This automatically removes common English
words like 'the', 'a', 'is', etc.

```

```

# These words don't carry much meaning for bug pattern mining, so we
# can ignore them.
tfidf_vectorizer = TfidfVectorizer(max_features=5000,
stop_words='english')

# Now, we use the vectorizer to transform our text data.
# 'fit_transform()' does two things in one step:
# 1. 'fit': It learns the vocabulary (all the unique words) from our
# data.
# 2. 'transform': It converts each log entry into a numerical vector
# based on the TF-IDF scores.
# The result is stored in a variable called 'X'.
X =
tfidf_vectorizer.fit_transform(df_log_based_bug_pattern_mining_dataset
['full_text'])

# We also need to define our target variable, which is what we want to
# predict.
# Our target is the 'error_type' column, and we'll store it in a
# variable called 'y'.
y = df_log_based_bug_pattern_mining_dataset['error_type']

print("\nFeature Matrix Shape ")
# This prints the size of our numerical dataset.
# The first number is the number of log entries (rows), and the second
# is the number of features (words).
# For example, (12500, 5000) means we have 12,500 logs and 5,000
# features.
print(f"Shape of X (features): {X.shape}")
print(f"Shape of y (target): {y.shape}")

Feature Matrix Shape
Shape of X (features): (12500, 546)
Shape of y (target): (12500,)

```

Model Training and Evaluation

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42, stratify=y)

# Initialize the Logistic Regression model.
# Logistic Regression is a simple but effective model for
# classification tasks.
# The 'class_weight='balanced'' parameter automatically adjusts for
# the class imbalance

```

```
# by giving more importance to the less frequent bug types.
model = LogisticRegression(class_weight='balanced',
solver='liblinear')

# Train the model.
# The '.fit()' method "trains" the model by learning the relationship
between the features (X_train)
# and the target labels (y_train).
print("Training the model")
model.fit(X_train, y_train)
print("Model training complete.")
```

Training the model
Model training complete.

```
# Model Evaluation
# Make predictions on the test set.
# The '.predict()' method uses the trained model to predict the error
type for new, unseen data (X_test).
y_pred = model.predict(X_test)

# Print the classification report.
# This report gives you a detailed look at your model's performance
for each bug type.
print("\nClassification Report")
print(classification_report(y_test, y_pred, zero_division=0))
```

Classification Report

	precision	recall	f1-score	support
ConnectionResetException	1.00	1.00	1.00	67
FileNotFoundException	1.00	1.00	1.00	113
IllegalArgumentException	1.00	1.00	1.00	77
IndexOutOfBoundsException	1.00	1.00	1.00	120
NoError	1.00	1.00	1.00	3006
NullPointerException	1.00	1.00	1.00	297
TimeoutException	1.00	1.00	1.00	70
accuracy			1.00	3750
macro avg	1.00	1.00	1.00	3750
weighted avg	1.00	1.00	1.00	3750

```
# Print the confusion matrix.
# This matrix shows you where your model was right and where it was
wrong.
print("\nConfusion Matrix")
cm = confusion_matrix(y_test, y_pred)
# We'll use pandas to make the confusion matrix more readable with row
and column labels.
```



```
cm_df = pd.DataFrame(cm, index=model.classes_, columns=model.classes_)
print(cm_df)
```

Confusion Matrix

	ConnectionResetException
FileNotFoundException \	
ConnectionResetException	67
0	
FileNotFoundException	0
113	
IllegalArgumentException	0
0	
IndexOutOfBoundsException	0
0	
NoError	0
0	
NullPointerException	0
0	
TimeoutException	0
0	

	IllegalArgumentException \
ConnectionResetException	0
FileNotFoundException	0
IllegalArgumentException	77
IndexOutOfBoundsException	0
NoError	0
NullPointerException	0
TimeoutException	0

	IndexOutOfBoundsException	NoError \
ConnectionResetException	0	0
FileNotFoundException	0	0
IllegalArgumentException	0	0
IndexOutOfBoundsException	120	0
NoError	0	3006
NullPointerException	0	0
TimeoutException	0	0

	NullPointerException	TimeoutException
ConnectionResetException	0	0
FileNotFoundException	0	0
IllegalArgumentException	0	0
IndexOutOfBoundsException	0	0
NoError	0	0
NullPointerException	297	0
TimeoutException	0	70

```
X_text = df_log_based_bug_pattern_mining_dataset['full_text']
y = df_log_based_bug_pattern_mining_dataset['error_type']
```

```
from sklearn.pipeline import Pipeline
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words='english')),
    ('classifier', LogisticRegression(solver='liblinear'))
])
```

```
param_grid = {
    'tfidf__max_features': [2000, 5000, 10000],
    'classifier__C': [0.1, 1, 10],
    'classifier__class_weight': ['balanced', None]
}
```

```
from sklearn.model_selection import GridSearchCV
grid_search = GridSearchCV(pipeline, param_grid, cv=5,
    scoring='f1_weighted', n_jobs=-1)
print("Starting Grid Search for optimal parameters")
grid_search.fit(X_text, y)
print("Grid Search complete.")
```

```
Starting Grid Search for optimal parameters
Grid Search complete.
```

```
# Print the best parameters and the best score
print("\nBest Parameters Found")
print(grid_search.best_params_)
print("\nBest F1-Score")
print(grid_search.best_score_)
```

```
Best Parameters Found
{'classifier__C': 0.1, 'classifier__class_weight': 'balanced',
 'tfidf__max_features': 2000}
```

```
Best F1-Score
1.0
```